Project Interim Report
On

# File Compression
# (Using Core Java & OOP Concepts

RU-100-01-00012

Object Oriented Programming With Java

SESSION: 2025-26


Submitted By

**Ashutosh Mishra**

RU-25-10324

Bachelor of Computer Application
& Engineering in association with Google

Under the Guidance of
Mr. Harsh Multani

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
RUNGTA INTERNATIONAL SKILLS UNIVERSITY
BHILAI, CG


(April 2026)

## ➢ Certificate

This is to certify that the project titled **"File Compression Simulator using OOP"** has been successfully completed by **Ashutosh Mishra** under the guidance of faculty.

**(Signature of HOD)**

---

## ➢ Acknowledgement

I would like to express my sincere gratitude to Mr. Harsh Multani and my college for providing me with the opportunity to work on this project. Their guidance and support helped me in understanding Object-Oriented Programming concepts practically. I also thank my friends and mentors for their valuable suggestions during the development of this project.

---

## ➢ Abstract

The **File Compression Simulator** is a Java application designed to demonstrate data optimization using **Object-Oriented Programming (OOP)**. It simulates the compression and decompression process through **Run-Length Encoding (RLE)**, a lossless technique that replaces consecutive repeating characters with a single character followed by its frequency. Rather than complex binary manipulation, this project focuses on **string manipulation** to make data reduction concepts accessible. The system uses a modular, class-based architecture where all logic is encapsulated within a `Compressor` class to ensure data security and maintainable design. By implementing `compress ()` and `decompress ()` methods, the project demonstrates the practical use of Java's `StringBuilder` and conditional logic. Ultimately, this simulator serves as an educational tool that allows users to observe data transformation while grounding them in core concepts like **loops, encapsulation, and algorithmic logic**.

---

## ➢ Introduction

**In modern educational and professional environments, managing data efficiently is a critical task. Manual handling of large datasets can lead to significant errors and inefficiency. Therefore, automated systems are required to simplify and optimize these processes.**

**The File Compression Simulator is designed using Core Java and follows Object-Oriented Programming (OOP) principles to address these needs. It allows users to input data strings, compress them into a shorter representation, and decompress them back to their original form. This project demonstrates the real-world implementation of several core programming concepts:**

- **Classes and Objects: Organizing logic into a modular Compressor class.**
- **Encapsulation: Protecting data integrity through private variables.**
- **Algorithmic Logic: Implementing Run-Length Encoding (RLE) for data reduction.**
- **String Manipulation: Utilizing efficient Java methods like StringBuilder.**

**By simulating these operations, the system improves understanding of programming logic, reduces manual effort, and ensures high accuracy in data handling.**

---

## ➢ Objectives

- To understand the concept of data compression and decompression.
- To implement string manipulation techniques in Java.
- To design a class-based system following OOP standards.
- To simulate real-world file processing logic.
- To strengthen problem-solving and logical thinking skills.

---

## ➢ Scope

**This project is designed for educational environments and small-scale data handling where basic compression functionality is required. It serves as a practical tool for students or developers to evaluate string optimization techniques. In the future, the system can be enhanced by:**

- **Adding Database Support: Integrating MySQL to store original and compressed strings for later retrieval.**
- **Creating a Graphical User Interface (GUI): Implementing Java Swing or JavaFX to replace the console-based input.**
- **Managing Multiple Records: Allowing the system to handle a batch of files or multiple user inputs simultaneously.**

- **File Input/Output: Moving from console-based string input to reading and writing directly from .txt or .java files.**
- **Advanced Logic: Handling spaces, special characters, and calculating the compression ratio to measure efficiency.**

---

## ➢ System Requirements

**To ensure the File Compression Simulator runs effectively, the following specifications are required:**

### Hardware:-

- **Computer/Laptop: A system with a minimum of 4GB RAM to handle the Java Development Kit (JDK) and IDE operations smoothly.**

### Software:-

- **Operating System: Windows, macOS, or Linux.**
- **Java JDK: Latest version of the Java Development Kit must be installed for compilation.**
- **IDE: An Integrated Development Environment such as VS Code, Eclipse, or IntelliJ IDEA for coding and debugging.**

---

## ➢ Methodology

The project follows these steps:

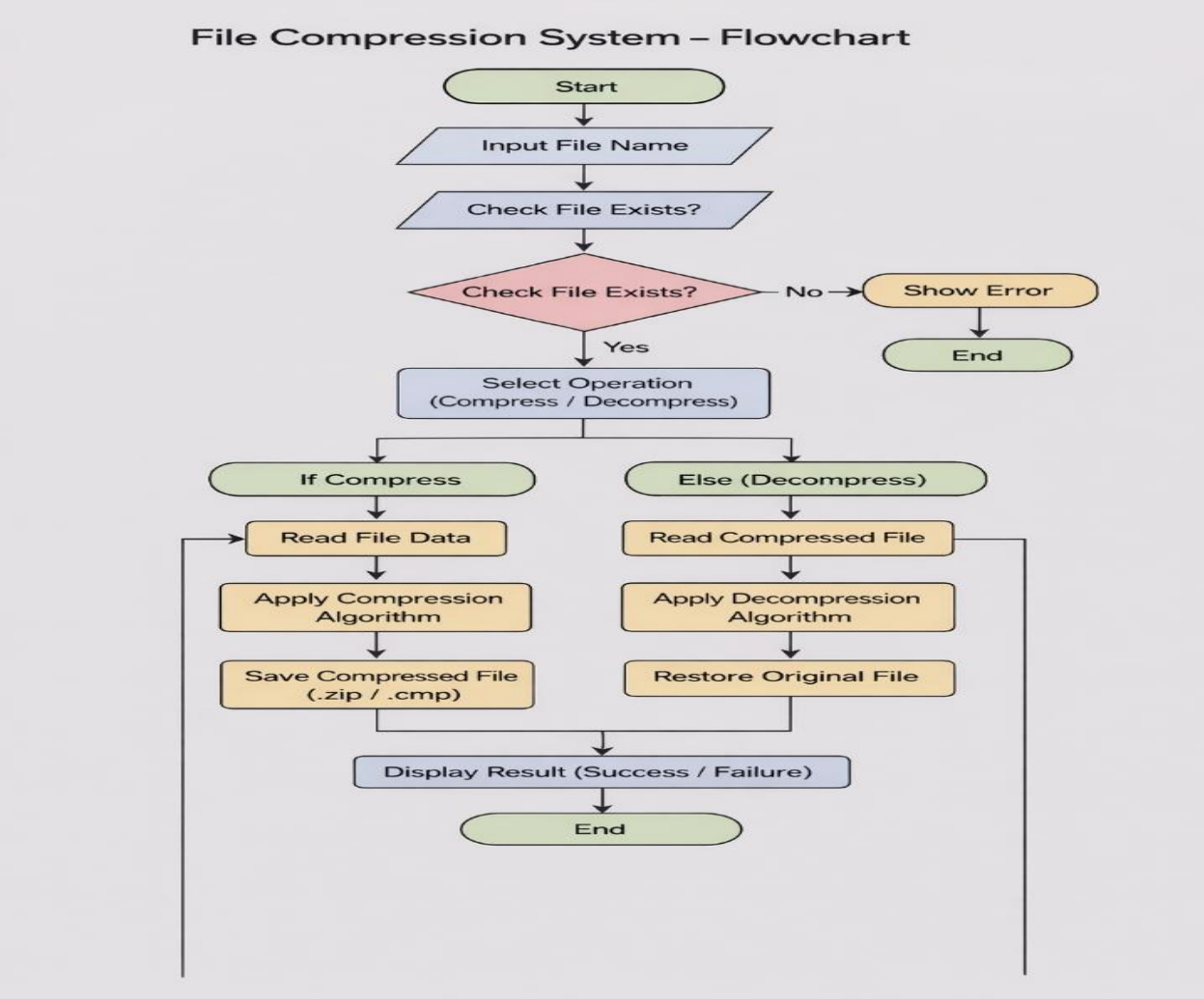
1. **Create Compressor Class:** Create a class named Compressor to handle operations.
2. **Implement compress() Method:** Use Run-Length Encoding (RLE) to count consecutive characters.
3. **Implement decompress() Method:** Reconstruct the original string from the compressed version.
4. **Display Results:** Show the original, compressed, and decompressed strings to validate correctness.

- # Flow Chart of the System

Based on the logic required for your project:

- **Start**: The program initializes and prompts the user for a string.
- **Input String**: The user enters a sequence of characters (e.g., `aaabbc`).
- **Initialize Loop**: The system sets up a `for` loop to traverse the string from the first character to the last.
- **Check Consecutive Characters**: A nested logic or counter checks if the next character is the same as the current one.
- **Build Compressed String**: The character and its final count are appended to a `StringBuilder` (e.g., `a3b2c1`) **Decompression Logic**: The system reverses the process to reconstruct the original string to ensure no data was lost.
- **Display Results**: The console outputs the Original, Compressed, and Decompressed strings for verification.

- **End**: The program terminates after displaying the results



## ➢ Implementation

The project is implemented using a primary class named `Compressor`, which encapsulates the logic for both data reduction and restoration. The system is designed to handle strings by treating them as sequences of characters, ensuring a modular and secure code structure.**The implementation details are as follows:**

- **Class Design**: The `Compressor` class contains private variables to store the `original String` and `compressedString`, adhering to the principle of encapsulation.
- **Compression Logic**: A dedicated method utilizes a `for` loop to traverse the input string, counting consecutive repeating characters and appending the results to a `StringBuilder` for memory efficiency.
- **Decompression Logic**: Another method identifies the character followed by its count to reconstruct the string exactly as it was originally entered.
- **Main Method (Testing)**: A driver class, such as `CompressionDemo`, creates an object of the `Compressor` class, takes user input via the `Scanner` class, and executes the functional methods.
- **Validation**: The program includes a check to ensure that the decompressed output matches the original input string to confirm data integrity.

---

- **Sample Input/Output**

Based on the project requirements, the system demonstrates the conversion of a raw string into a compressed format and back again to ensure data integrity.

- **Input**: Enter String: aaaabbc
- **Output**:
  - **Compressed String**: a4b2c1
  - **Decompressed String**: aaaabbc.

---

## ➢ Future Enhancements

To evolve the **File Compression Simulator** from a basic console application into a more robust tool, the following enhancements are planned:

- **Database Integration**: Adding **MySQL** support to store history of original and compressed strings for later retrieval.

- **Graphical User Interface (GUI)**: Creating a more user-friendly interface using **Java Swing** or JavaFX to replace the terminal-based input.
- **Advanced Data Handling**:
    - Adding support for **multiple student/user records** to be processed in batches.
    - Implementing **File I/O** to read from and write to actual physical files instead of just console input.
- **Reporting and Analysis**:
    - Exporting compression reports in **PDF format**.
    - Including **Compression Ratio calculations** to measure the efficiency of the RLE logic.
- **Robust Logic**: Handling special characters, spaces, and invalid compressed strings to prevent program crashes.

---

## ➢ Conclusion

The **File Compression Simulator** successfully demonstrates the use of Object-Oriented Programming concepts in Java. It provides an efficient way to simulate data optimization and reconstruction automatically. The project improves coding skills and helps in understanding real-world applications of programming.

---

## ➢ References


- Java Documentation
- Online tutorial
- OOP textbooks